Лабораторная работа №4

Вычисление наибольшего общего делителя

11 Марта 2026 - Сунь Маосин

2026

Российский университет дружбы народов, Москва, Россия

Цель работы

Основная цель

В данной лабораторной работе я реализовал четыре алгоритма нахождения наибольшего общего делителя: классический алгоритм Евклида, бинарный алгоритм Евклида, а также их расширенные версии. Основная цель — изучить принципы работы этих алгоритмов и сравнить их эффективность.

Реализация алгоритмов

Алгоритм 1: Алгоритм Евклида

Классический алгоритм Евклида основан на последовательном делении с остатком. В цикле большее число заменяется остатком от деления на меньшее до тех пор, пока одно из чисел не станет равным нулю.

Код реализации

```
def euclid_simple(a, b):
    """
    Simple Euclidean algorithm
    Input: integers a, b
    Output: d = GCD(a, b)
    """
    while a != 0 and b != 0:
        if a >= b:
```

Алгоритм 2: Бинарный алгоритм Евклида

Бинарный алгоритм использует свойства чётности чисел и заменяет операцию деления на более быстрые операции сдвига и вычитания, что делает его эффективнее для компьютерной реализации.

Код реализации

```
def euclid_binary(a, b):
    """
    Binary Euclidean algorithm
    Input: integers a, b
    Output: d = GCD(a, b)
    """
    g = 1

    # Remove common factors of 2
    while a % 2 == 0 and b % 2 == 0:
        a //= 2
        b //= 2
        g *= 2

    u, v = a, b

    # Main loop
    while u != 0:
        # Remove factors of 2 from u
        while u % 2 == 0:
            u //= 2

        # Remove factors of 2 from v
```

Алгоритм 3: Расширенный алгоритм Евклида

Расширенный алгоритм Евклида позволяет не только найти НОД, но и вычислить коэффициенты x и y из соотношения Безу: $ax + by = d$, где $d = (a, b)$.

Код реализации

```
def euclid_extended(a, b):
    """
    Extended Euclidean algorithm
    Input: integers a, b
    Output: (d, x, y) such that d = GCD(a, b) and a*x + b*y = d
    """
    if a == 0:
        return (b, 0, 1)

    d, x1, y1 = euclid_extended(b % a, a)
    x = y1 - (b // a) * x1
    y = x1
```

Алгоритм 4: Расширенный бинарный алгоритм Евклида

Данный алгоритм объединяет преимущества бинарного алгоритма с возможностью нахождения коэффициентов линейной комбинации без использования операции деления.

Код реализации

```
def euclid_binary_extended(a, b):
    '''
    Extended binary Euclidean algorithm
    Input: integers a, b
    Output: (d, x, y) such that d = GCD(a, b) and a*x + b*y = d
    '''
    g = 1

    # Remove common factors of 2
    while a % 2 == 0 and b % 2 == 0:
        a //= 2
        b //= 2
        g *= 2

    u, v = a, b
    A, B, C, D = 1, 0, 0, 1

    # Main loop
    while u != 0:
        # Remove factors of 2 from u
        while u % 2 == 0:
            u //= 2
            if A % 2 == 0 and B % 2 == 0:
                A //= 2
                B //= 2
            else:
                A = (A + b) // 2
                B = (B - a) // 2

        # Remove factors of 2 from v
        while v % 2 == 0:
            v //= 2
            if C % 2 == 0 and D % 2 == 0:
                C //= 2
                D //= 2
            else:
                C = (C + b) // 2
                D = (D - a) // 2
```

Тестирование всех алгоритмов

Тестирование всех алгоритмов

Для проверки корректности работы всех алгоритмов были использованы тестовые примеры из задания. Программа выводит результаты каждого алгоритма и проверяет их соответствие.

```
============================================================
LABORATORY WORK No. 4
Testing GCD Algorithms
============================================================

a = 12345, b = 24690
----------------------------------------
Simple Euclidean: GCD = 12345
Binary Euclidean: GCD = 12345
Extended Euclidean: GCD = 12345, x = 1, y = 0
  Verification: 12345*1 + 24690*0 = 12345
Extended Binary Euclidean: GCD = 12345, x = 12345, y = -6172
  Verification: 12345*12345 + 24690*-6172 = 12345
✓ All algorithms give the same result

a = 12345, b = 54321
----------------------------------------
Simple Euclidean: GCD = 3
Binary Euclidean: GCD = 3
Extended Euclidean: GCD = 3, x = 3617, y = -822
  Verification: 12345*3617 + 54321*-822 = 3
Extended Binary Euclidean: GCD = 3, x = -14490, y = 3293
  Verification: 12345*-14490 + 54321*3293 = 3
✓ All algorithms give the same result

a = 12345, b = 12541
----------------------------------------
Simple Euclidean: GCD = 1
Binary Euclidean: GCD = 1
Extended Euclidean: GCD = 1, x = 4159, y = -4094
  Verification: 12345*4159 + 12541*-4094 = 1
Extended Binary Euclidean: GCD = 1, x = 4159, y = -4094
  Verification: 12345*4159 + 12541*-4094 = 1
✓ All algorithms give the same result

a = 91, b = 105
----------------------------------------
Simple Euclidean: GCD = 7
Binary Euclidean: GCD = 7
Extended Euclidean: GCD = 7, x = 7, y = -6
  Verification: 91*7 + 105*-6 = 7
Extended Binary Euclidean: GCD = 7, x = 52, y = -45
```

Вывод

Вывод

В ходе лабораторной работы я успешно реализовал четыре алгоритма нахождения наибольшего общего делителя. Эксперимент подтвердил, что классический алгоритм Евклида наиболее прост в реализации, бинарный алгоритм эффективнее на низком уровне за счёт использования сдвигов, а расширенные алгоритмы позволяют находить коэффициенты линейной комбинации, что важно для решения диофантовых уравнений и криптографических приложений.